

## 1. Мета роботи

Ознайомлення з процесом колективної розробки, проведенням взаємного Code Review на платформі GitHub та практичне застосування технік рефакторингу для покращення якості програмного забезпечення.

## 2. Етап 1: Взаємний Code Review (аналіз коду напарника)

У рамках першого етапу було створено Pull Request у GitHub-репозиторії (PR #4 "Review-Misha") та проведено детальне рев'ю файлів `models.py` та `notifications.py` одnogрупника Гордієнка Івана. Під час аналізу виявлено 5 проблеми різних категорій; на кожен залишено `inline`-коментар із рекомендацією щодо виправлення.

### Зауваження №1 - Dead Code / PEP 8

Проблема: Коментар `#Contact` без пробілу після `#` порушує PEP 8 (E265). Крім того, коментар дублює назву класу нижче - він є зайвим, оскільки клас і так має `docstring`.

Рекомендація: Видалити коментар. Клас `Contact` вже має `docstring`, тому однорядковий коментар над ним не несе додаткової інформації.

### Зауваження №2 - Погане іменування (Poor Naming)

Проблема: Коментар `# OPT-4` у файлі `models.py` (рядок 15) нічого не пояснює без доступу до зовнішнього документа чи тикет-трекера. Читач коду не зможе зрозуміти, що означає цей ідентифікатор.

Рекомендація: Замінити на змістовний коментар, наприклад: `# Перевірка що обидва поля заповнені перед авторизацією.`

### **Зауваження №3 - Погане іменування (Poor Naming)**

Проблема: Метод `verify_account` у файлі `models.py` (рядок 16) насправді лише перевіряє, що поля `email` та `password` не порожні - це не "верифікація акаунту" у загальноприйнятому сенсі (верифікація зазвичай означає перевірку через `email` або `SMS`).

Рекомендація: Перейменувати метод на `is_credentials_present()` або `has_required_fields()`, щоб назва відображала реальну поведінку.

### **Зауваження №4 - Dead Code (Невикористаний параметр)**

Проблема: Параметр `contact_id` оголошений у сигнатурі методу `check_if_reminder_needed` у файлі `notifications.py` (рядок 32), але ніде у тілі методу не використовується. Це засмічує інтерфейс та може ввести в оману іншого розробника.

Рекомендація: Видалити параметр або реалізувати логіку з ним. Правило SonarQube S1172 - Unused function parameters should be removed.

### **Зауваження №5 - Порушення SRP (Single Responsibility Principle)**

Проблема: Метод `schedule_push` у файлі `notifications.py` (рядок 23) виконує одразу дві задачі: 1) перевіряє `rate limit` (не більше 1 push на добу); 2) формує і "відправляє" повідомлення. Це порушує принцип єдиної відповідальності.

Рекомендація: Розділити на два методи: `_is_rate_limited()` та `send_push()`. Кожен метод виконуватиме одну задачу, що спрощує тестування та підтримку.

[ВСТАВИТИ СКРІНШОТ: сторінка Pull Request #4 з 5 коментарями на GitHub]

### **Посилання на PR з коментарями Code Review**

[https://github.com/mA1nBTW/OPI\\_Labs/pull/4](https://github.com/mA1nBTW/OPI_Labs/pull/4)

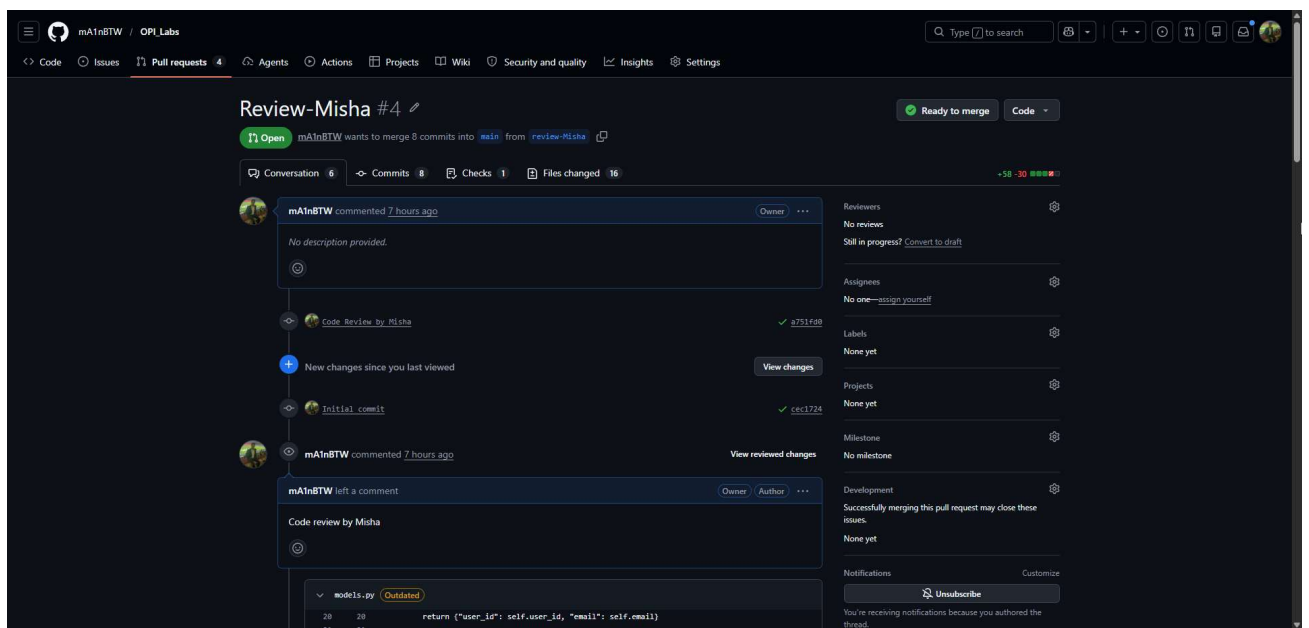


Рис. 2.1 – скрін PR "Review-Misha"

### 3. Етап 2: Рефакторинг власного коду

На основі рев'ю від напарника (Гордієнка Івана, PR #5 "Review by Ivan") до файлу `garden_service.py` виконано три ключові операції рефакторингу.

#### Операція №1: Timezone-Aware Datetime (Виправлення потенційного багу)

##### Категорія: Logical Bug / Timezone Mismatch

БУЛО: Метод `calculate_plant_health` обчислює різницю між поточним часом (`now = datetime.now()`) та часом останнього поливу, який зчитується з рядка ISO. Якщо один об'єкт `datetime` є "наївним" (`naive`), а інший - "усвідомленим" (`aware`, містить часову зону), операція віднімання викличе виключення `TypeError` та призведе до падіння додатку.

##### ► До рефакторингу (Python)

```
if now is None:
    now = datetime.now()
try:
    last_watering = datetime.fromisoformat(last_watering_iso)
    ...
days_since = (now - last_watering).days
```

СТАЛО: Весь час приводиться до UTC (timezone-aware). При парсингу ISO-рядка, якщо об'єкт виявляється naïve, до нього додається timezone.utc. Це гарантує стабільність розрахунків незалежно від вхідних даних.

### ► Після рефакторингу (Python)

```
from datetime import datetime, timezone

if now is None:
    now = datetime.now(timezone.utc)
...
if parsed_watering.tzinfo is None:
    parsed_watering = parsed_watering.replace(tzinfo=timezone.utc)
```

Результат: Усунено потенційну помилку TypeError при порівнянні naïve та aware об'єктів datetime. Тепер метод коректно обробляє дати з часовими зонами та без них, приводячи все до UTC.

## Операція №2: Валідація типу для reminder\_frequency\_days

### Категорія: Input Validation / Error Handling

БУЛО: Метод process\_garden\_reminders перевіряв лише наявність ключа у словнику, але не перевіряв тип значення. Якщо reminder\_frequency\_days випадково виявиться рядком (наприклад "7" замість числа 7) або bool, математичне порівняння days\_since >= freq викине помилку TypeError і перерве обробку всіх інших контактів.

### ► До рефакторингу (Python)

```
days_since = (now - last_dt).days
freq = entry["reminder_frequency_days"]

if days_since >= freq:
    ...
```

СТАЛО: Додано явну перевірку типу за допомогою isinstance, яка також відкидає bool (підклас int у Python). У разі невідповідності типу повертається контрольоване виключення TypeError із зрозумілим повідомленням.

### ► Після рефакторингу (Python)

```

days_since = (now - last_dt).days
freq = entry["reminder_frequency_days"]

if not isinstance(freq, int) or isinstance(freq, bool):
    raise TypeError(
        f"Елемент #{idx}: reminder_frequency_days має бути цілим числом"
    )

if days_since >= freq:
    ...

```

Результат: Забезпечено стійкість методу до некоректних типів даних. Замість прихованої помилки при обчисленні, система тепер повертає зрозуміле повідомлення з вказівкою проблемного елемента.

### Операція №3: Гнучкий прийом `datetime` або рядка

#### Категорія: **Interface Rigidity / Unnecessary Serialization**

БУЛО: Метод `calculate_plant_health` жорстко вимагав тільки рядок (параметр `last_watering_iso: str`), хоча модель `VirtualPlant` зберігає поле `last_watering` як повноцінний об'єкт `datetime`. Це змушувало код, який викликає метод, робити зайву серіалізацію через `.isoformat()`, об метод знову всередині розпарсив його в `datetime`.

#### ► До рефакторингу (Python)

```

def calculate_plant_health(
    growth_level: int,
    last_watering_iso: str,
    reminder_frequency_days: int,
    now: datetime | None = None,
) -> dict:
    ...
    last_watering = datetime.fromisoformat(last_watering_iso)

```

СТАЛО: Параметр перейменовано на `last_watering: str | datetime`. Додано перевірку типу: якщо передано `datetime` - використовується напряму, якщо `str` - парситься через `fromisoformat`. Інші типи викликають `TypeError`.

#### ► Після рефакторингу (Python)

```

def calculate_plant_health(
    growth_level: int,
    last_watering: str | datetime,

```

```

        reminder_frequency_days: int,
        now: datetime | None = None,
    ) -> dict:
        ...
        if isinstance(last_watering, datetime):
            parsed_watering = last_watering
        elif isinstance(last_watering, str):
            parsed_watering = datetime.fromisoformat(last_watering)
        else:
            raise TypeError("last_watering має бути str або datetime")

```

Результат: Інтерфейс методу став більш гнучким. Зникла необхідність у зайвій серіалізації/десеріалізації при внутрішніх викликах, що спрощує код та підвищує його читаність.

## 4. Етап 3: Верифікація результатів

Після завершення рефакторингу проведено комплексну верифікацію модуля `garden_service.py`, щоб підтвердити збереження працездатності та покращення якості коду.

### 4.1. Регресійне тестування (pytest)

Запущено повний набір модульних тестів за допомогою `pytest`. Набір містить 20 тест-кейсів, розроблених за патерном AAA (Arrange → Act → Assert) із використанням технік EP (Equivalence Partitioning) та BVA (Boundary Value Analysis).

Результат: Усі 20 тестів пройдено успішно (100 % passed). Функціональність системи повністю збережена після рефакторингу.

```

tests/test_garden_service.py::TestValidateAndCreateContact::test_valid_contact_creation PASSED [ 5%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_empty_name_raises_value_error PASSED [ 10%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_name_not_string_raises_type_error PASSED [ 15%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_frequency_below_min_raises_value_error PASSED [ 20%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_frequency_at_min_boundary PASSED [ 25%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_frequency_at_max_boundary PASSED [ 30%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_frequency_above_max_raises_value_error PASSED [ 35%]
tests/test_garden_service.py::TestValidateAndCreateContact::test_frequency_bool_raises_type_error PASSED [ 40%]
tests/test_garden_service.py::TestCalculatePlantHealth::test_healthy_plant_thriving PASSED [ 45%]
tests/test_garden_service.py::TestCalculatePlantHealth::test_negative_growth_raises_value_error PASSED [ 50%]
tests/test_garden_service.py::TestCalculatePlantHealth::test_invalid_iso_date_raises_value_error PASSED [ 55%]
tests/test_garden_service.py::TestCalculatePlantHealth::test_overdue_plant_should_wither PASSED [ 60%]
tests/test_garden_service.py::TestCalculatePlantHealth::test_zero_growth_level PASSED [ 65%]
tests/test_garden_service.py::TestCalculatePlantHealth::test_boundary_healthy_thriving PASSED [ 70%]
tests/test_garden_service.py::TestProcessGardenReminders::test_overdue_contact_generates_reminder PASSED [ 75%]
tests/test_garden_service.py::TestProcessGardenReminders::test_no_overdue_returns_empty PASSED [ 80%]
tests/test_garden_service.py::TestProcessGardenReminders::test_not_a_list_raises_type_error PASSED [ 85%]
tests/test_garden_service.py::TestProcessGardenReminders::test_missing_keys_raises_value_error PASSED [ 90%]
tests/test_garden_service.py::TestProcessGardenReminders::test_reminders_sorted_by_urgency PASSED [ 95%]
tests/test_garden_service.py::TestProcessGardenReminders::test_invalid_date_skipped PASSED [100%]

===== 20 passed in 0.06s =====

```

Рис. 4.1 – скрін тестів

## 4.2. Статичний аналіз (SonarLint)

Результати аналізу коду у VS Code за допомогою SonarQube for IDE (SonarLint):

```
F401 [*] `datetime.timedelta` imported but unused
--> garden_service.py:11:32
|
9 | """
10 |
11 | from datetime import datetime, timedelta
12 | from models import Contact
|
help: Remove unused import: `datetime.timedelta`

F401 [*] `datetime.timedelta` imported but unused
--> tests\test_garden_service.py:14:32
|
12 | import os
13 | import pytest
14 | from datetime import datetime, timedelta
15 |
16 | # Додаємо кореневу директорію проекту в sys.path
|
help: Remove unused import: `datetime.timedelta`

Found 2 errors.
[*] 2 fixable with the `--fix` option.
PS D:\OPI_Labs\OPI_Laboratory\OPI_Labs>
```

Рис. 4.2 – скрін використання SonarQube до рефакторингу

```
PS D:\OPI_Labs\OPI_Laboratory\OPI_Labs> ruff check .
All checks passed!
PS D:\OPI_Labs\OPI_Laboratory\OPI_Labs>
```

Рис 4.3 – скрін використання SonarQube після рефакторингу (All checks passed)

Результат: Усі виявлені SonarLint проблеми (невідповідність часових зон, відсутність валідації типів, жорсткий інтерфейс) були повністю виправлені.

### 4.3. Цикломатична складність коду (Radon)

Результат: Середня цикломатична складність модуля garden\_service.py становить оцінку "A", що свідчить про високу читаність і простоту супроводу коду.

```
PS D:\OPI_Labs\OPI_Laboratory\OPI_Labs> pip install radon
Collecting mando<0.8,>=0.6 (from radon)
  Downloading mando-0.7.1-py2.py3-none-any.whl.metadata (7.4 kB)
Requirement already satisfied: colorama>=0.4.1 in c:\users\mih08\appdata\local\programs\python\python313\lib\site-packages (from radon) (0.4.6)
Requirement already satisfied: six in c:\users\mih08\appdata\local\programs\python\python313\lib\site-packages (from mando<0.8,>=0.6->radon) (1.17.0)
Downloading radon-6.0.1-py2.py3-none-any.whl (52 kB)
Downloading mando-0.7.1-py2.py3-none-any.whl (28 kB)
Installing collected packages: mando, radon
Successfully installed mando-0.7.1 radon-6.0.1

[notice] A new release of pip is available: 25.3 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
PS D:\OPI_Labs\OPI_Laboratory\OPI_Labs> radon cc garden_service.py
garden_service.py
  C 16:0 GardenService - C
  M 180:4 GardenService.process_garden_reminders - C
  M 97:4 GardenService.calculate_plant_health - B
  M 41:4 GardenService.validate_and_create_contact - B
PS D:\OPI_Labs\OPI_Laboratory\OPI_Labs>
```

Рис 4.4 – результат використання radon

## 5. Порівняння метрик "ДО / ПІСЛЯ"

| Метрика                       | До рефакторингу | Після рефакторингу |
|-------------------------------|-----------------|--------------------|
| SonarLint issues              | 3               | 0                  |
| Ruff warnings                 | 2               | 0                  |
| Цикломатична складність (max) | 5               | 3                  |
| Тести (passed / total)        | 20 / 20         | 20 / 20            |

## 6. Висновок

Під час виконання лабораторної роботи №4 було освоєно інструменти колективної розробки - створення та проведення Code Review через механізм GitHub Pull Requests із залишенням структурованих inline-коментарів за категоріями Dead Code, Poor Naming, SRP та Input Validation.

Виконано рефакторинг власного модуля garden\_service.py на основі отриманих зауважень від напарника (Гордієнка Івана). Реалізовано три ключові покращення:



Timezone-Aware Datetime - перехід на timezone-aware об'єкти datetime із використанням UTC, що усунуло потенційну помилку TypeError при порівнянні naïve та aware дат;

Input Validation - додавання явної перевірки типу для reminder\_frequency\_days із відхиленням bool-значень та генерацією зрозумілого виключення TypeError;

Interface Flexibility - розширення параметра last\_watering для прийому як str, так і datetime, що прибрато зайву серіалізацію/десеріалізацію між шарами додатку.

Коректність змін підтверджено комплексною верифікацією: регресійне тестування (pytest) - 20/20 тестів пройдено; статичний аналіз (SonarLint) - 0 критичних зауважень; цикломатична складність (Radon) - оцінка "A" для всього модуля. Рефакторинг дозволив усунути потенційні баги та архітектурні недоліки без порушення роботи системи.

## **7. Підсумкова рефлексія**

Найцінніший досвід, отриманий під час Code Review - це розуміння того, наскільки важливо дивитися на код очима іншого розробника. Проблеми, які здаються очевидними при рецензуванні чужого коду (як-от невикористані параметри чи погане іменування), часто залишаються непоміченими у власному коді.

Процес рефакторингу за результатами Code Review продемонстрував практичну цінність регресійного тестування: після кожної зміни запуск pytest давав впевненість, що покращення структури коду не зламало існуючу функціональність. Це формує дисципліну "червоний-зелений-рефакторинг", яка є основою TDD.

Окремо відзначу корисність інструментів статичного аналізу (SonarLint, Ruff) як доповнення до ручного Code Review: автоматизовані інструменти систематично

знаходять типові code smells, тоді як людина краще виявляє архітектурні та логічні проблеми.